# BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

**(An Autonomous Institution, Affiliated to VTU, Belagavi)**

**Seminar (22MCA402)**

Report On

## CI/CD: Automating the Software Development Lifecycle

## MASTER OF COMPUTER APPLICATIONS

BY

**VAIBHAV KARBHANTNAL**

**1BY23MC097**

**2024-25**

**Even Semester**

**BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT**

**(An Autonomous Institution, Affiliated to VTU Belagavi)**
**Avalahalli, Doddaballapur Main Road, Bengaluru - 560064**

**DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS**

# CERTIFICATE

This is to certify that **Vaibhav Karbhantnal** bearing **1BY23MC097** has satisfactorily completed the Seminar – 22MCA402 with **CI/CD: Automating the Software Development Lifecycle** in the academic year 2024-25 submitted in partial fulfillment of the requirements of the 4th semester of Master of Computer Applications.

**Signature of the Candidate**

**Signature of the Guide**
**Prof. Ashwitha K**
Assistant Professor
Department of MCA
BMSIT&M
Bengaluru

**Signature of the HOD**
**Dr. M Sridevi**
Assistant Professor &
Head Department of MCA
BMSIT&M
Bengaluru

| Marks | |
|---|---|
| **Maximum** | **Obtained** |
| **100** | |

# BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

**(An Autonomous Institution, Affiliated to VTU Belagavi)**
**Avalahalli, Doddaballapur Main Road, Bengaluru - 560064**

## DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

## VISION

To emerge as a leading department in computer applications, producing skilled professionals equipped to deliver sustainable solutions.

## MISSION

Facilitate effective learning environment through quality education, industry interaction with orientation towards research, critical thinking and entrepreneurial skills.

## PROGRAMME EDUCATIONAL OBJECTIVES

**PEO 1:** Excel in IT career by developing sustainable solutions that drive industry growth and societal progress.

**PEO 2:** Adapt themselves to evolving domain requirements.

**PEO 3:** Exhibit leadership skills and progress in their chosen career path.

## PROGRAM OUTCOMES

**PO1:** Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

**PO2**: Identify, review, formulate and analyse problems for primarily focusing on customer requirements using critical thinking frameworks.

**PO3:** Design, develop and investigate problems with an innovative approach for solutions incorporating ESG/SDG goals.

**PO4:** Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

**PO5:** Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups using methodologies such as agile.

**PO6**: Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.

**PO7:** Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware.

**PO8:** Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

**Rubrics for Seminar Presentation Assessment (out of 70 marks) =**

**Rubrics for Seminar Report Assessment (out of 20 marks) =**

**Adhering to submission deadlines (out of 10 marks) =**

**Total Marks (Out of 100 marks) =**

# BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

**(An Autonomous Institution, Affiliated to VTU, Belagavi)**

## DEPARTMENT OF MCA

**Evaluation Sheet - Seminar (22MCA402)**

**Batch:** 2023-25 **Academic Year:** 2024-25 **Sem:** IV

**Name : Vaibhav Karbhantnal** **USN : 1BY23MC097**

**Title : CI/CD: Automating the Software Development Lifecycle**

| Evaluators | Communication Skills | Societal Concern | Topic Relevance | Self Learning | Time Utilization | Q&A | Signature | Remarks |
|---|---|---|---|---|---|---|---|---|
| Max Marks | 15 | 10 | 10 | 15 | 10 | 10 | | |
| Evaluator 1 | | | | | | | | |
| Evaluator 2 | | | | | | | | |
| Average | | | | | | | | |
| | Marks awarded for presentation (Max : 70 Marks ) | | | | | | | |
| | Marks Awarded for Seminar Report (Max: 20 Marks) | | | | | | | |
| | Adhering to submission deadlines (Max: 10 Marks) | | | | | | | |
| | Total Marks out of 100 | | | | | | | |

**Guide** **HOD, MCA**

# BMS INSTITUTE OF TECHNOLOGY AND MANAGEMENT

**(An Autonomous Institution, Affiliated to VTU, Belagavi )**

## DEPARTMENT OF MCA

## Report Evaluation Rubrics

| | Excellent (5) | V. Good (4) | Good (3) | Satisfactory (2) | Poor (1) | Final Score |
|---|---|---|---|---|---|---|
| **Purpose and Objective** | The purpose and objective of the report is made clear, and the report addresses the objective(s) in a focused and logical manner. | The purpose and objective of the report is made clear, and the report addresses the objective(s). | Documented well but with slight ambiguity. | Purpose and objectives are stated ambiguously. | The report does not clearly address the objective(s). | |
| **Grammar& Spelling** | Very few spelling errors, correct punctuation, grammatically correct, complete. | Occasional lapses in spelling, punctuation, grammar, but not enough to seriously consider. | Grammatical mistakes not corrected. | Sentences are not framed properly and with a few spelling mistakes. | Numerous spelling errors, non-existent or incorrect punctuation, and/or severe errors in grammar that interfere with understanding. | |
| **Report Format** | All required elements of the report are present and completed. | All required elements of the report are present and completed to an extent. | All required elements of the report are present but some are not relevant. | All required elements of the report are provided but in a haphazard manner. | Key elements of the report are not provided. Overall presentation of the document is not to a professional standard. | |
| **Plagiarism Check** | Plagiarism below 10%. | Plagiarism between 10% and 15%. | Plagiarism between 15% and 20%. | Plagiarism between 20% and 25%. | Plagiarism more than 25%. | |
| | | | | | **Total** | |

# Table of Content

# 1. INTRODUCTION

In the rapidly evolving field of software engineering, the pressure to deliver high-quality software faster, more frequently, and with fewer defects has driven the adoption of more dynamic and automated development practices. As organizations aim to remain competitive and agile in a digital-first world, the need for reliable, scalable, and repeatable development and deployment processes has become critical. Traditional methodologies such as the Waterfall model, characterized by its rigid sequential phases (requirements, design, implementation, verification, and maintenance), have proven inadequate in meeting the demands of modern software development—where flexibility, iteration, and speed are paramount.

To address these shortcomings, the software development landscape has shifted towards **Agile**, **DevOps**, and automation-centric practices. One of the most transformative methodologies to emerge in this context is **CI/CD**, which stands for **Continuous Integration** and **Continuous Delivery/Deployment**. CI/CD is not just a set of tools, but a cultural and engineering philosophy that emphasizes frequent, incremental code changes and rapid feedback loops. It automates the software release process, enabling development teams to deliver changes to users quickly and safely.

**Continuous Integration (CI)** is the practice of frequently merging small batches of code changes into a central repository. Each integration triggers automated build processes and test suites, ensuring that changes are validated early and integrated smoothly. CI encourages developers to maintain clean, working code at all times, thereby reducing integration headaches, catching bugs early, and ensuring high code quality.

**Continuous Delivery (CD)** builds upon CI by automating the release process, making it possible to deploy validated changes to staging or pre-production environments at any time with minimal manual effort. This means that software is always in a release-ready state. The more advanced stage, **Continuous Deployment**, automates this even further—every code change that passes automated tests is deployed directly to production. This ensures rapid delivery and feedback, making it especially valuable in environments where business needs and customer expectations change frequently.

The CI/CD pipeline typically includes stages such as source control management, automated building, testing (unit, integration, system), artifact management, deployment, and monitoring. Each stage helps ensure that the software moves through development to deployment with minimal friction and maximum reliability. This paradigm shift also supports **Test-Driven Development (TDD)**, **Infrastructure as Code (IaC)**, and **observability-first operations**, bringing a high degree of consistency and transparency to the entire software lifecycle.

CI/CD is central to **DevOps**, a cultural and technical movement that breaks down the silos between development and operations teams, emphasizing collaboration, communication, and shared ownership of the delivery pipeline. With CI/CD, developers and operations personnel can work together to shorten feedback loops, reduce manual work, improve stability, and deploy code with confidence.

Modern tools have significantly accelerated the adoption of CI/CD. Open-source platforms like **Jenkins**, **Travis CI**, and **GitLab CI** pioneered automated integration and delivery. Cloud-based solutions like **CircleCI**, **GitHub Actions**, **Azure Pipelines**, and **Google Cloud Build** have made CI/CD accessible to teams of all sizes by abstracting infrastructure concerns and offering seamless integrations with code hosting platforms. Moreover, these tools provide rich ecosystems of plugins and reusable components, enabling custom workflows tailored to specific use cases.

The rise of **containerization (e.g., Docker)** and **orchestration (e.g., Kubernetes)** has further enhanced CI/CD pipelines by providing consistent environments for development, testing, and deployment. This allows for immutable infrastructure practices, reducing discrepancies between environments and increasing deployment confidence. Similarly, in areas such as **mobile development**, **machine learning**, and **infrastructure automation**, specialized CI/CD workflows are emerging to handle unique challenges like multi-platform builds, model retraining, and state management.

As CI/CD continues to evolve, it is reshaping not only how software is built and deployed, but also how teams collaborate, innovate, and respond to customer needs. By automating repetitive tasks, reducing errors, and promoting continuous feedback, CI/CD enables organizations to deliver better software faster—and to do so sustainably.

This report explores the historical development, architectural components, applications, tools, and open challenges of CI/CD. Through real-world use cases and insights into current trends, it aims to highlight the transformative impact of CI/CD on the software industry and underscore its role as a fundamental practice in modern software engineering.

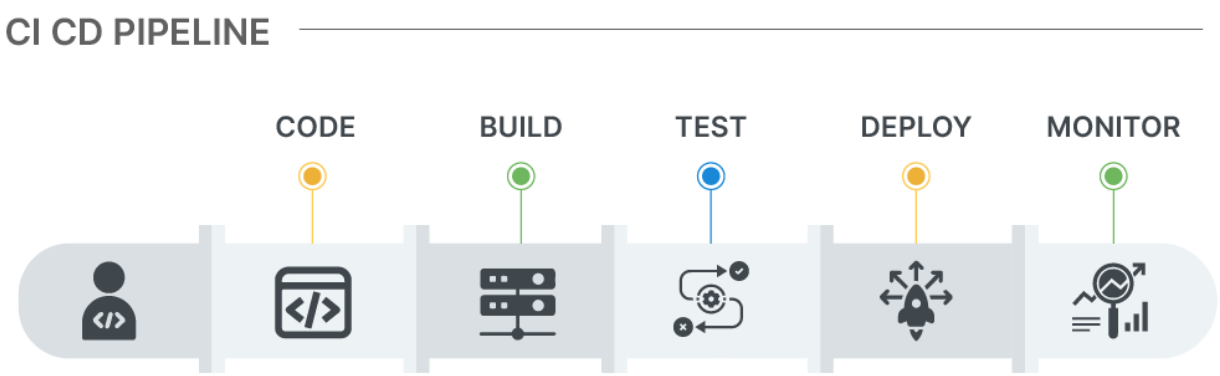



Fig1.1 CICD Pipline

# 2. HISTORY

The concept of Continuous Integration (CI) dates back to the early 1990s when Grady Booch first introduced it in his book *Object-Oriented Analysis and Design with Applications*. However, it was the Extreme Programming (XP) movement in the late 1990s that popularized CI as a core practice. XP proponents like Kent Beck and Ron Jeffries emphasized frequent code integration and practices such as the "ten-minute build," which encouraged rapid builds and testing using unit tests.

The early 2000s saw the first wave of CI tools with the release of **CruiseControl** in 2001 by ThoughtWorks, followed by **Hudson** in 2005, created by Kohsuke Kawaguchi at Sun Microsystems. After a legal dispute, Hudson was renamed **Jenkins** in 2011, which remains a widely used CI server today.

Before CI/CD, software releases were infrequent, manual, and prone to error. Developers often built and released software from their local machines, leading to inconsistencies and bugs. Centralized build systems and automated testing helped address these challenges by improving frequency and consistency.

The 2010s brought about the **second generation of CI/CD** with the rise of cloud computing. Platforms like **TravisCI** and **CircleCI** offered CI-as-a-service, tightly integrated with cloud-based code repositories like GitHub. These platforms simplified setup and improved performance, helping CI/CD gain widespread adoption.

Soon after, cloud providers such as AWS, Google, and Microsoft introduced their own native CI/CD services — **CodeBuild**, **Cloud Build**, and **Azure DevOps**, respectively — particularly targeting enterprise users.

The **third generation** of CI/CD emerged with the seamless integration of CI tools into version control platforms. **GitLab** was a pioneer in embedding CI/CD into its Git hosting in 2015, followed by **GitHub Actions** in 2018. GitHub Actions gained massive popularity due to its reusability, ease of configuration, and growing community ecosystem.

Today, CI/CD is a foundational practice in modern DevOps workflows. Its evolution has drastically changed software development — from manual releases and rare updates to automated pipelines and frequent deployments — ensuring higher quality, faster feedback, and continuous innovation.

# 3. AREA OF APPLICATION

Continuous Integration and Continuous Delivery (CI/CD) have emerged as fundamental pillars in modern software engineering, enabling organizations to automate, streamline, and scale their development and deployment processes. By introducing faster feedback loops, reducing manual intervention, and enforcing code quality through automation, CI/CD enhances both the speed and reliability of software delivery. The following are key areas where CI/CD practices are actively transforming industries:

## 3.1.E-Commerce

In the high-stakes world of e-commerce, where customer expectations and competition are both intense, businesses must innovate and deploy updates rapidly without compromising stability. CI/CD allows online retailers to:

- **Deploy updates multiple times per day** without affecting ongoing user transactions.
- Use **automated test suites** to validate changes in payment processing, search algorithms, inventory management, and customer portals.
- Implement **canary releases and A/B testing**, which allow new features to be rolled out gradually and tested with specific user groups.
- Handle **seasonal spikes in traffic** (e.g., during Black Friday or holiday sales) with confidence, thanks to performance testing and scalable deployment processes.
- **Quickly rollback** faulty deployments, minimizing customer-facing disruptions.

In essence, CI/CD transforms e-commerce platforms into agile, customer-responsive ecosystems, capable of delivering new features, offers, and enhancements faster than ever [1].

## 3.2.Financial Services

The financial sector—including banks, insurance companies, trading platforms, and fintech startups—demands extremely high standards of security, compliance, and reliability. CI/CD pipelines in this domain are engineered with these requirements in mind:

- **Security testing and auditing tools** (e.g., static code analysis, dynamic analysis, and compliance scanning) are deeply integrated into the CI/CD pipeline.
- Every code change goes through **automated checks** for compliance with financial regulations (such as PCI-DSS or SOX).
- CI/CD helps ensure **consistent deployment** of applications across complex infrastructures including cloud, hybrid, and on-premise systems.
- **Microservices architecture**, powered by CI/CD, enables modular updates to customer accounts, transaction services, or analytics without affecting other systems.

By leveraging CI/CD, financial organizations can confidently release updates faster, improve service availability, and reduce time-to-market for digital innovations [2].

## 3.3.Healthcare

Healthcare software deals with sensitive patient data and life-critical applications, making software quality and compliance absolutely essential. CI/CD provides:

- **Automated testing for regulatory compliance** with standards such as HIPAA, GDPR, and FDA guidelines.
- Continuous delivery of **updates to EHR (Electronic Health Record) systems**, appointment scheduling apps, and telemedicine portals.
- Integration of **performance, accessibility, and usability tests**, ensuring services are inclusive and reliable under various conditions.
- Capability to quickly deploy **critical bug fixes** without delays that might otherwise endanger patient care.

Hospitals and healthcare software vendors are increasingly adopting CI/CD to support real-time monitoring systems, patient care platforms, and remote diagnostics—bringing software innovation to medical services [3].

## 3.4.Gaming Industry

The gaming industry is highly dynamic, often releasing frequent updates to maintain player engagement and fix bugs rapidly. CI/CD in game development supports:

- **Continuous asset integration** (textures, audio, maps, etc.) along with code, ensuring cohesive builds.
- **Cross-platform build automation**, enabling games to be tested and deployed across PC, consoles, and mobile environments from a single pipeline.
- **Multiplayer game server deployment**, where backend services can be updated with minimal latency or downtime.
- Delivery of **live updates, DLCs, and seasonal content**, enabling studios to keep games fresh and aligned with community expectations.
- **Crash and telemetry data integration**, allowing real-time feedback loops for stability and performance improvements.

CI/CD enhances responsiveness and innovation in game development, which is crucial in a fast-paced and feedback-driven market [4].

## 3.5.Machine Learning and Artificial Intelligence

Unlike traditional applications, ML/AI systems rely not only on code but also on constantly changing data and models. CI/CD enables:

- **Version control of datasets, models, and pipelines**, ensuring reproducibility.
- Automation of **model training and evaluation** using fresh datasets.
- Integration of **model drift detection**, automatically triggering retraining when model accuracy degrades.

- Continuous deployment of **ML APIs, dashboards, or embedded AI systems**, ensuring insights are always up-to-date.
- Efficient **collaboration between data scientists and software engineers**, bridging gaps through shared automation practices.

This is particularly valuable in applications such as fraud detection, recommendation systems, personalized marketing, and intelligent automation [5].

## 3.6.Embedded Systems and IoT

Embedded systems and Internet of Things (IoT) devices pose unique deployment challenges due to hardware constraints, environmental variability, and the scale of device fleets. CI/CD helps tackle these challenges by:

- Automating the **compilation of firmware** across multiple processor architectures and board configurations.
- Running **hardware-in-the-loop (HIL)** tests and simulations to ensure firmware behaves correctly under real-world conditions.
- Supporting **Over-the-Air (OTA) updates**, which enable remote and secure firmware deployment to edge devices without physical access.
- Ensuring **backward compatibility** across multiple device generations.
- Managing device configuration, diagnostics, and rollback through **centralized orchestration tools**.

Industries such as automotive, agriculture, industrial automation, smart homes, and wearables benefit significantly from CI/CD practices that bring software-like agility to hardware development [6].

# 4. OPEN CHALLENGES

While CI/CD brings numerous benefits to the software development lifecycle, its adoption is not without challenges. One major issue lies in the complexity of setting up and maintaining a robust pipeline. Integrating various tools, managing configurations, and ensuring compatibility across environments demands a high level of expertise and continuous attention as projects evolve.

Another significant challenge is achieving effective test automation and adequate coverage. Automated testing is the backbone of CI/CD, yet writing comprehensive and reliable tests for diverse scenarios can be difficult. Incomplete coverage may allow bugs to slip through, whereas lengthy test suites can slow down the entire pipeline.

Security is also a persistent concern. With automation extending into deployment processes, there is a heightened risk of exposing sensitive data or introducing vulnerabilities if pipelines are not properly secured. Ensuring security at every stage requires dedicated strategies and regular monitoring.

Beyond technical hurdles, cultural and organizational resistance can impede CI/CD adoption. Shifting to an automated and iterative development approach often demands changes in team structure, workflows, and mindset. Teams accustomed to traditional development cycles may struggle to adapt, affecting the overall effectiveness of the CI/CD strategy.

Additionally, many organizations face difficulty in integrating CI/CD practices with legacy systems. These systems, not originally designed for frequent updates or automation, often lack the flexibility required to support modern CI/CD processes, resulting in additional complications during integration.

# 5. USES CASES

CI/CD practices are widely implemented across various domains to automate, streamline, and accelerate development workflows. One of the most common use cases is in applications that require **frequent feature releases**, such as **e-commerce platforms** and **social media services**. These applications benefit greatly from the rapid and reliable delivery enabled by CI/CD pipelines, which ensure that new features can be rolled out quickly without sacrificing quality or user experience.

Another important use case is the **deployment of hotfixes and security patches**. In production environments, where downtime or bugs can have significant consequences, CI/CD allows development teams to push critical fixes swiftly. With automated testing and deployment in place, such patches can be validated and released with minimal delay, reducing system vulnerabilities and ensuring business continuity.

CI/CD is also essential in **microservices architectures**, where each service is developed, tested, and deployed independently. By maintaining separate pipelines for each microservice, CI/CD enables teams to deploy updates to individual services without affecting the entire system. This enhances scalability, modularity, and fault isolation, making the architecture more resilient and manageable.

**Infrastructure as Code (IaC)** is another domain where CI/CD shines. DevOps teams leverage pipelines to automate cloud infrastructure provisioning and updates using tools like **Terraform**, **Ansible**, or **AWS CloudFormation**. This ensures that infrastructure changes are consistent, traceable, and free from manual errors, thereby improving both security and agility in system operations.

In the field of **machine learning (ML)**, CI/CD plays a critical role in automating model **training**, **testing**, and **deployment**. By integrating these steps into a pipeline, teams can ensure **reproducibility of experiments**, maintain **model performance over time**, and deploy updated models quickly in response to new data or business needs. This is especially valuable in industries like finance, healthcare, and e-commerce, where ML models are core to personalization, diagnostics, or fraud detection.

## Additional Use Cases

### 5.1. Mobile App Development

CI/CD pipelines enable seamless development and deployment of mobile apps across multiple platforms (iOS, Android). With automation tools like **Fastlane**, builds are automatically tested, signed, and distributed to app stores or beta testing platforms. This reduces human error, speeds up release cycles, and ensures consistent delivery across devices.

### 5.2. Continuous Testing and QA Automation

Modern QA processes heavily rely on CI/CD pipelines to execute **automated unit, integration, and UI tests**. This ensures bugs are caught early, features are validated across environments, and regression issues are minimized. Tools like **Selenium**, **Appium**, and **JUnit** are integrated into pipelines to support test automation and reporting.

## 5.3.DevSecOps: Security Integration in CI/CD

Organizations are embedding **security checks** within CI/CD pipelines to enforce policies like static code analysis, vulnerability scanning, and secrets detection before deployment. This "**shift-left**" approach ensures security is addressed early in the development cycle, reducing the cost and impact of vulnerabilities in production.

## 5.4.Edge Computing and IoT Deployments

In edge and IoT environments, CI/CD is used to manage **firmware updates** and **remote deployments** across distributed devices. Pipelines ensure that updates are **tested, signed, and delivered securely** to devices with minimal disruption. This is especially relevant in smart cities, industrial automation, and wearable technologies.

## 5.5.SaaS and Cloud-Native Application Delivery

In academic environments, CI/CD is used to ensure **research reproducibility**, especially in computational fields. Pipelines manage the code, data, and experiment configurations, allowing other researchers to validate findings and build on them confidently.

## 5.6.Enterprise Business Applications

Large organizations running ERP, CRM, or HR systems use CI/CD to automate **custom module updates**, **plugin integration**, and **UI changes** without interfering with critical business workflows. This reduces deployment risk and enables continuous improvement in business processes.

# 6. CI/CD PIPELINE: TOOLS AND WORKFLOW

A CI/CD pipeline is a set of automated processes that allow software development teams to build, test, and deploy code reliably and efficiently. The pipeline ensures that code changes go through multiple stages of verification before reaching production, reducing human error and accelerating delivery.

## Pipeline Stages

A typical CI/CD pipeline includes the following stages:

1. **Source Code Management (SCM):**
   The pipeline is triggered when changes are pushed to a version control system like GitHub, GitLab, or Bitbucket. This ensures all changes are tracked and integrated from multiple developers.
2. **Build Stage:**
   In this stage, the source code is compiled and built into executable files. Build tools like Maven, Gradle, or Webpack are used, depending on the language and framework. If the build fails, the pipeline halts, preventing further execution.
3. **Automated Testing:**
   Unit tests, integration tests, and sometimes UI tests are executed to verify that the application functions as expected. This step ensures that newly integrated code does not break existing functionality.
4. **Artifact Storage:**
   Successful builds are packaged and stored as artifacts using tools like JFrog Artifactory or Nexus Repository. These artifacts are later used in the deployment stage.
5. **Deployment:**
   The application is deployed to a staging or production environment. Continuous delivery stops here, requiring manual approval for production deployment. Continuous deployment automates this step as well.
6. **Monitoring and Feedback:**
   After deployment, tools like Prometheus, Grafana, or ELK Stack monitor performance and errors. Feedback loops are essential to detect issues early and initiate rollback if needed.

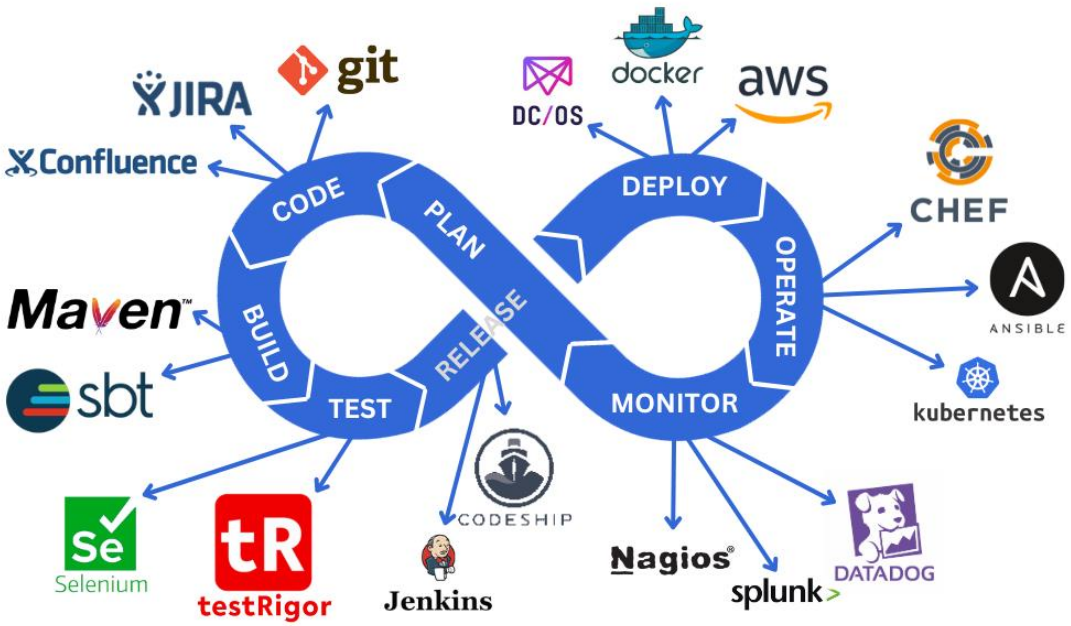



Fig6.1 CICD Stages

## Popular Tools in the CI/CD Ecosystem

- **CI Tools:** Jenkins, GitHub Actions, GitLab CI/CD, CircleCI, Travis CI
- **Build Tools:** Maven, Gradle, Ant
- **Testing Frameworks:** JUnit, Selenium, PyTest, Mocha
- **Artifact Repositories:** JFrog Artifactory, Nexus
- **Deployment Tools:** Docker, Kubernetes, Ansible, Helm
- **Monitoring Tools:** Prometheus, Grafana, ELK Stack

## Workflow Summary

The workflow begins when a developer commits code to a shared repository. The CI/CD system automatically:

1. Detects the commit.
2. Pulls the latest code.
3. Builds and tests the application.
4. Stores the tested build as an artifact.
5. Deploys the application.
6. Monitors its behaviour in production.

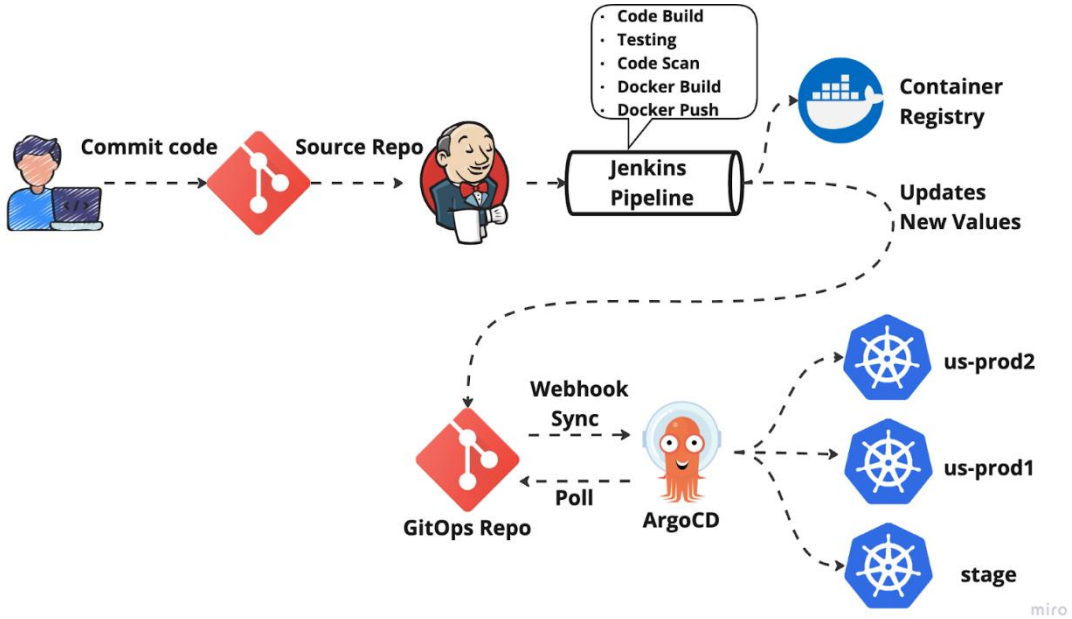



Fig 6.2 CICD Workflow

# 7. CONCLUSION

CI/CD has become an essential part of modern software development by streamlining the processes of integration, testing, and deployment. It enables teams to release high-quality software faster and more reliably through automation and continuous feedback. Although implementing CI/CD can pose challenges such as managing tool complexity and ensuring adequate test coverage, its benefits in improving development efficiency, reducing errors, and accelerating delivery far outweigh the drawbacks. As development practices evolve, CI/CD will continue to play a critical role in building scalable, maintainable, and robust software systems.

# 8. REFERENCES

[1] The Importance of CI/CD Integration in Software Development and Quality Assurance, Sayge.in,2023. https://sayge.in/the-importance-of-ci-cd-integration-in-software-development-and-quality-assurance
[2] CI/CD in Financial Services, Sayge.in, 2023. https://sayge.in/the-importance-of-ci-cd-integration-in-software-development-and-quality-assurance
[3] CI/CD Applications in Healthcare, Sayge.in, 2023. https://sayge.in/the-importance-of-ci-cd-integration-in-software-development-and-quality-assurance
[4] CI/CD in Gaming Industry, Sayge.in, 2023. https://sayge.in/the-importance-of-ci-cd-integration-in-software-development-and-quality-assurance
[5] Breaking Silos: Unifying DevOps and MLOps into a Unified Software Supply Chain, TechRadar.com, 2023.
[6] https://www.techradar.com/pro/breaking-silos-unifying-devops-and-mlops-into-a-unified-software-supply-chain
[7] Why CI/CD is Essential for Modern Software Development, Debug.school, 2023.
[8] https://www.debug.school/rakeshdevcotocus_468/why-cicd-is-essential-for-modern-software-development-24p4
[9] Sayge.in, "Challenges of Setting Up CI/CD Pipelines," 2023. https://sayge.in/ci-cd-challenges-and-best-practices
[10] TechBeacon.com, "Why Test Automation Is Key to CI/CD Success," 2023. https://techbeacon.com/devops/why-test-automation-key-cicd-success
[11] DevOps.com, "Securing Your CI/CD Pipeline: Best Practices," 2023. https://devops.com/securing-your-ci-cd-pipeline-best-practices/
[12] Atlassian.com, "How to Overcome Resistance to DevOps and CI/CD Adoption," 2023. https://www.atlassian.com/devops/adoption-resistance
[13] InfoWorld.com, "Legacy Systems and CI/CD: The Integration Challenge," 2023. https://www.infoworld.com/article/3576857/legacy-systems-and-ci-cd.html

# 9. Plagiarism Report

DrillBit

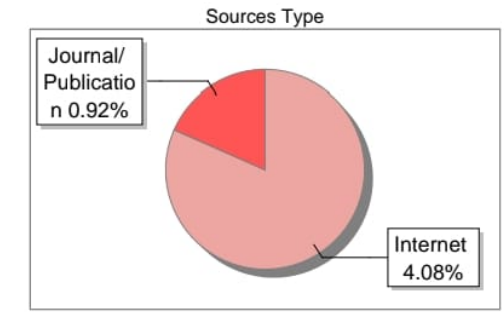
The Report is Generated by DrillBit Plagiarism Detection Software

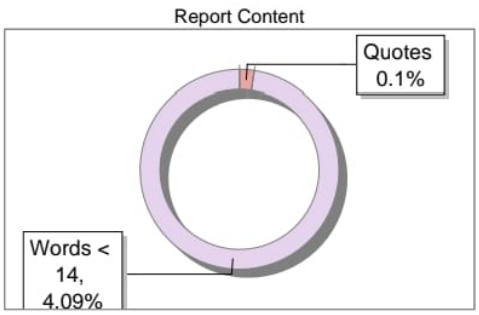
***Submission Information***

| | |
|---|---|
| Author Name | VAIBHAV KARBHANTNAL |
| Title | CICD |
| Paper/Submission ID | 3699546 |
| Submitted by | anitha.ks@bmsit.in |
| Submission Date | 2025-05-30 11:35:58 |
| Total Pages, Total Words | 7, 2076 |
| Document type | Project Work |

***Result Information***

Similarity **5 %**




***Exclude Information***

| | |
|---|---|
| Quotes | Not Excluded |
| References/Bibliography | Excluded |
| Source: Excluded < 14 Words | Not Excluded |
| Excluded Source | **0 %** |
| Excluded Phrases | Not Excluded |

***Database Selection***

| | |
|---|---|
| Language | English |
| Student Papers | Yes |
| Journals & publishers | Yes |
| Internet or Web | Yes |
| Institution Repository | Yes |

A Unique QR Code use to View/Download/Share Pdf File